Automated task-dataset generation from the Help Desk knowledge base

Automated task-dataset generation from the Help Desk knowledge base

This pipeline turns a real, LINKED help-desk knowledge base into a task-specific training dataset, using a LOCAL model (Ollama) to generate samples WITH reasoning traces, then trains and evaluates one task.

What it does, end to end: survey the data and confirm which fields are relevant (and which would leak the answer); then iterate ticket by ticket, pulling each ticket's linked context and generating one tailored training sample with a reasoning trace; keep the data balanced and the reasoning chains varied in length; mix in a little general-reasoning and tool-calling data so the model keeps those abilities; train a small LoRA adapter; and accept it only if a two-axis evaluation shows the task improved without the model regressing on general benchmarks.

The concrete task it trains and measures is help-desk resolution prediction (see "The task" below). It reuses the proven spine from track_a / track_b (define -> seed/gold -> grounded synthetic generation -> judge -> dedup/decontaminate/balance -> LoRA -> two-axis eval).

The corpus (linked tables)

Raw files in this folder (read-only inputs; see FEATURES.md, EXAMPLE.md, db.png):

- `issues.csv` — one row per issue; `id` is the unique issue key (66,691 issues). Metadata + aggregated workflow time-in-state (`wf_<state>` seconds, `wfe_<state>` pass counts) + the resolution outcome.
- `issues_snapshot.csv` — per-assignee TURN snapshots; join on `id`.
- `issues_change_history.csv` — raw assignee/status change log; `issueid` -> `id`.
- `sample_utterances.csv` — masked conversation text; `issueid` -> `id` (only ~360 appraised issues have text).
- `issues_snapshot_sample.xlsx` — human appraisal scores (reserved for a later task).

`link.py` joins these by issue id; `serialize.py` renders one issue's linked context into a compact, leakage-aware prompt block.

The task

Binary resolution prediction: **Done vs Won't Do**, from the structured linked context (metadata + workflow time-in-state + pre-resolution handling path). The model reasons in a `<think>` block then states the label. Metric: accuracy + macro-F1 (macro because Done dominates). The verified label comes from the data, so only the reasoning trace is synthetic (the grounding rule). The conversation text is sparse for this task (6 Won't Do issues have text) so it is not used here; `serialize.py` is text-aware so an appraisal/text task can be added later cheaply.

Run order

Generation runs LOCAL on Ollama; only training runs on Kaggle. Run from this folder.

```
# 0. one-time: local model
ollama pull qwen2.5:3b-instruct
ollama serve

# 1. survey the data, propose relevant vs leakage fields (the configure
step)
../../.venv/bin/python phase1_seed/survey.py
#    -> review data/field_survey.json, confirm the leakage excludes, set
"confirmed": true

# 2. build the sacred gold set + seed + training id pool (split by issue
id)
../../.venv/bin/python phase1_seed/build_seed.py

# 3. generate reasoning traces per ticket (varied length), judge, filter,
assemble
../../.venv/bin/python phase2_synthetic/gen_reasoning.py
../../.venv/bin/python phase2_synthetic/judge.py
../../.venv/bin/python phase2_synthetic/filter.py

# 4. mix in general-reasoning + tool-calling rehearsal (~75/25)
../../.venv/bin/python phase2_synthetic/mix_rehearsal.py

# 5. train the control and the real run (GPU; see kaggle_notebook/ for the
cloud path)
../../.venv/bin/python phase3_train/train.py --data data/seed.jsonl      -
-name seed
../../.venv/bin/python phase3_train/train.py --data data/train_mix.jsonl -
-name seed-synth

# 6. evaluate on both axes and line them up
../../.venv/bin/python eval/evaluate.py --name base
../../.venv/bin/python eval/evaluate.py --name seed       --adapter
phase3_train/lora-seed
../../.venv/bin/python eval/evaluate.py --name seed-synth --adapter
phase3_train/lora-seed-synth
../../.venv/bin/python eval/compare.py
```

Smoke-test each generation step first with `--limit` (e.g. `gen_reasoning.py --limit 5`) since local generation is call-heavy.

What each piece does

- `survey.py` -> `data/field_survey.json`: the local model proposes which fields are predictive vs which LEAK the outcome (the final status, resolution date, and outcome-coupled workflow states like rejected/cancelled/done). Merged with a safe default exclude list; a human confirms. Downstream steps refuse to run until confirmed.
- `build_seed.py`: reserves gold FIRST, split by issue id (no leakage), balanced Done/Won't Do; small verified seed with short honest traces; a disjoint training id pool.
- `gen_reasoning.py`: per ticket, fixes the verified label and asks the local model for a reasoning trace that justifies it from the fields; rejects any trace whose answer is not the real label; varies trace length (short/medium/long).
- `judge.py`: faithfulness gate, keeps traces that follow from the fields and validly

support the label (score >= 4).
- `filter.py`: decontaminate by issue id vs gold, drop near-duplicate contexts, balance across classes and trace-length buckets, assemble `train_synth.jsonl` (= seeds + kept).
- `mix_rehearsal.py`: append ~25% rehearsal (GSM8K general reasoning + a small function-calling slice) -> `train_mix.jsonl`, to preserve those abilities.
- `train.py`: QLoRA on Qwen2.5-0.5B-Instruct, assistant-only loss, `max_len` 1408.
- `evaluate.py`: axis 1 = accuracy + macro-F1 on gold; axis 2 = three probe sets (`sentinel` general, `reasoning` step-by-step, `tools` function-calling). `compare.py` lines up base vs seed vs seed-synth with deltas.

Disciplines enforced

- Gold reserved first and split by issue id; decontamination by id before training.
- Leakage controlled by the survey+confirm gate plus serializer excludes (no final status/resolution/date, no outcome-coupled workflow states, terminal statuses trimmed from the handling path).
- Verified labels from the data; only the reasoning is generated (grounding rule).
- Two-axis eval with acceptance up front: task macro-F1 should rise while the sentinel / reasoning / tool probes hold.

Cloud training

The 4 GB local GPU is fine for generation (Ollama) but cramped for training the long context + trace. `kaggle_notebook/track_c_kaggle.md` has paste-ready cells to train on a free Kaggle T4: upload `data/{seed.jsonl,train_mix.jsonl,gold.jsonl}` and `eval/*.jsonl` as a dataset, train, evaluate, download the adapter.

Test the trained model

Predict a ticket's resolution interactively (reasoning trace + label):

```
../../.venv/bin/python run_trained.py
# ticket id (blank = next gold)> 1004364
```

A ticket id builds the context from the linked tables (needs a confirmed field_survey.json); a blank line runs the next held-out gold ticket. Flags: `--adapter <path>`, `--issue-id <id>` for a one-shot run.